

PHASE 5

Project Development

Project: **FitTrack-AI** – Personal Intelligent Fitness & Workout Tracker

The supplied repository implements the **FitTrack-AI** backend using Node.js and Express.js with MongoDB/Mongoose. The backend is designed with a modular architecture split across configuration, models, middleware, controllers, routes, and AI intelligence services.

1. Technologies Used

Technology	Role & Application
Node.js	Asynchronous server-side JavaScript execution environment.
Express.js	Minimalist RESTful API framework for routing and middleware execution.
MongoDB & Mongoose	NoSQL document database and Object Data Modeling (ODM) library for user, workout, and nutrition data persistence.
JSON Web Tokens (JWT)	Stateless authentication mechanism using <code>jsonwebtoken</code> .
bcryptjs	Salted hashing algorithm for secure user password storage.
Google Gemini AI	Integrated via <code>@google/generative-ai</code> for personalized fitness insights, workout generation, and macro advice.
Fitness Data API	Integrates with external fitness trackers/calculators or falls back to internal BMR/TDEE calculation algorithms.
CORS & dotenv	Cross-Origin Resource Sharing handling and environment variable management.
Postman	Comprehensive end-to-end API testing and collection validation.

2. Main Modules Implemented

Module	Implementation & Responsibilities
Application	<code>app.js</code> configures CORS, JSON body parsing, API routes, 404 handler, and centralized error middleware.
Server	<code>server.js</code> establishes MongoDB connection via Mongoose and boots the Express application on the target port.
Authentication	

Module	Implementation & Responsibilities
	<code>authController.js</code> manages user registration, authentication, and profile fetching. <code>authMiddleware.js</code> verifies JWT token on secured endpoints.
User Model	Stores account credentials (name, email, hashed password), user fitness goals, weight, height, age, activity level, and timestamps.
Workout Model	Stores logged fitness sessions: exercise type, duration, sets, reps, estimated calories burned, user reference, and index.
Workout Controller	Executes CRUD operations (add, list, update, delete) for fitness activity logging with strictly enforced user ownership checks.
Fitness Tracker Service	Calculates calorie expenditure, daily active minutes, and retrieves workout data or uses deterministic metric estimations.
AI Workout & Diet Service	Generates personalized workout routines, recovery suggestions, and tailored dietary advice through Gemini AI or rule-based fallback engines.
Routes	Exposes RESTful endpoints for authentication, workout management, fitness metrics, and AI recommendations.

3. API Endpoint Set

Method / Endpoint	Access	Purpose
POST /api/auth/register	Public	Register a new user account and receive a JWT authentication token.
POST /api/auth/login	Public	Authenticate existing user credentials and return a active JWT token.
GET /api/auth/profile	JWT	Retrieve profile information and physical metrics of the logged-in user.
POST /api/workouts	JWT	Log a new workout session (exercise type, sets, reps, duration, intensity).
GET /api/workouts	JWT	Fetch user's workout history and fitness logs filtered by date or activity type.
PUT /api/workouts/:id	JWT	Update an existing workout entry after verified ownership check.
DELETE /api/workouts/:id	JWT	Remove a specific workout log from the database.
GET /api/fitness/metrics	Public	Return standard baseline metrics, BMR/TDEE calculations, and burn rates.

Method / Endpoint	Access	Purpose
POST /api/ai/workout-plan	JWT	Generate custom workout schedules based on target goals and workout logs.
POST /api/ai/nutrition-summary	JWT	Provide personalized nutritional insights and daily macro advice via Gemini AI.

4. External Service and Fallback Logic

Resilient Architecture & Fault Tolerance

When `FITNESS_API_KEY` is missing or unreachable, the workout service dynamically falls back to deterministic energy expenditure estimations based on standard MET (Metabolic Equivalent of Task) values and user body weight. Similarly, when `GEMINI_API_KEY` is unavailable or generation fails, the AI engine triggers local rule-based fitness algorithms. This guarantees seamless offline development, automated testing, and reliable execution without strict cloud API dependencies.

5. Running the Project

To launch and execute the backend application locally, follow these standard npm commands:

```
# 1. Install all required dependencies
npm install

# 2. Start application in development mode (with nodemon reload)
npm run dev

# 3. Production-style startup execution
npm start

# Default server runs on http://localhost:5000 unless specified via PORT environment variable
```